

Month 2 – React + Next.js: Detailed Exercises (Weeks 5-8, Days 29-56)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 14 Sep 2026 – 11 Oct 2026 (Days 29-56)

 **Your slots (IST):** Mon-Fri **7:30-8:30pm** & **9:00-10:00pm** · Sat-Sun **9:00am-12:00pm, 3:00-5:30pm, 6:00-8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import `calendar/mern-mastery-study.ics`

Roadmap: [ROADMAP.md](#) · Theory: [03-react-complete.md](#), [04-nextjs-complete.md](#) · Index: [week-05-08-exercises.md](#)

WEEK 5 – React Fundamentals (Days 29–35)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Exercise W5-1: Vite + React + TypeScript Setup

Problem statement

Scaffold a Vite React TypeScript project with ESLint, folder structure (`components/` , `hooks/` , `types/`), and dev server running.

Learning objective

Modern React toolchain; strict TS in frontend.

Prerequisites

- Theory: [03-react-complete.md §1–3](#)

Step-by-step approach

1. `npm create vite@latest react-lab -- --template react-ts` .
2. `cd react-lab && npm install` .
3. Configure path aliases in `vite.config.ts` if desired (`@/`).
4. Remove boilerplate; add `App.tsx` placeholder.
5. Run `npm run dev` ; verify hot reload.
6. Add `.gitignore` , initial commit to `mern-mastery/month-02/` .
7. Document scripts in README.

Hints

1. Vite uses native ESM — fast HMR.
2. Keep components in PascalCase files.
3. Enable strict in tsconfig if not default.

Exact Answer

- App loads at `http://localhost:5173` with HMR working
- Folder structure: `src/components/` , `src/hooks/` , `src/types/`
- `npm run build` succeeds with strict TypeScript

Complete Solution

```
npm create vite@latest react-lab -- --template react-ts
cd react-lab && npm install
mkdir -p src/components src/hooks src/types
```

```
// vite.config.ts - optional path alias
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import path from "path";

export default defineConfig({
  plugins: [react()],
  resolve: { alias: { "@": path.resolve(__dirname, "./src") } },
});
```

```
// src/App.tsx
export default function App() {
  return <h1>React Lab Ready</h1>;
}
```

Expected output / acceptance criteria

- App loads at <http://localhost:5173>
- TS compiles without errors

Where to practice

- Vite: <https://vitejs.dev/guide/>
- React: <https://react.dev/learn/installation>

Common mistakes

- Wrong Node version; use 18+.
- Mixing default vs named exports inconsistently.

80 LPA Interview Twist

Meta asks you to compare Vite vs Webpack vs Turbopack build pipelines and explain when you'd eject to a monorepo with shared packages — toolchain decisions matter at 80 LPA.

Interview connection

"Walk me through bootstrapping a React app" — Vite vs CRA vs Next.

Time estimate: 45 minutes

Exercise W5-2: Counter, Toggle, Greeting Components

Problem statement

Build three components demonstrating props and state:

- **Counter** — increment/decrement, optional step prop
- **Toggle** — boolean visibility switch
- **Greeting** — accepts **name**, time-based message optional

Learning objective

useState, props typing, unidirectional data flow.

Prerequisites

- Theory: [03-react-complete.md §4](#)

Step-by-step approach

1. Define prop interfaces for each component.
2. Counter: `const [count, setCount] = useState(0)`.
3. Toggle: `const [on, setOn] = useState(false)` conditional render children.
4. Greeting: presentational — no state required.
5. Compose all in `App.tsx` with layout.
6. Add basic CSS modules or Tailwind.
7. Ensure keys not needed yet (no lists).

Hints

1. Never mutate state directly — always setter.
2. Props read-only; lift state up if siblings share data later.
3. Type optional props with `?`.

Exact Answer

- Counter with `step={5}` increments/decrements by 5
- Toggle shows/hides children on click
- Greeting renders `"Good morning, {name}"` (or time-based variant)

Complete Solution

```
import { useState, ReactNode } from "react";

interface CounterProps { step?: number; }
export function Counter({ step = 1 }: CounterProps) {
  const [count, setCount] = useState(0);
  return (
    <div>
      <button onClick={() => setCount(c => c - step)}>-</button>
      <span>{count}</span>
      <button onClick={() => setCount(c => c + step)}>+</button>
    </div>
  );
}

interface ToggleProps { children: ReactNode; }
export function Toggle({ children }: ToggleProps) {
  const [on, setOn] = useState(false);
  return (
    <div>
      <button onClick={() => setOn(o => !o)}>{on ? "Hide" : "Show"}</button>
      {on && children}
    </div>
  );
}

interface GreetingProps { name: string; }
export function Greeting({ name }: GreetingProps) {
  const hour = new Date().getHours();
  const time = hour < 12 ? "Good morning" : hour < 18 ? "Good afternoon" : "Good evening";
  return <h2>{time}, {name}!</h2>;
}
```

Expected output / acceptance criteria

- Counter respects `step={5}` ; Toggle shows/hides content; Greeting renders personalized text.

Where to practice

- React Learn: <https://react.dev/learn/state-a-components-memory>
- Local: `month-02/week-05/components/`

Common mistakes

- Calling `setState` in render loop.
- Missing prop types causing runtime undefined.

80 LPA Interview Twist

Google frontend asks you to refactor these into compound components with Context, or explain React 19 `use()` for async props — testing component API design beyond basics.

Interview connection

Controlled vs uncontrolled components foundation.

Time estimate: 60 minutes

Exercise W5-3: Todo List with useState

Problem statement

Todo app: add task, delete, toggle complete. Local state only.

Learning objective

List rendering, keys, immutable state updates.

Prerequisites

- Theory: [03-react-complete.md §7 Lists](#)

Step-by-step approach

1. State: `Todo[]` with `{ id, text, done }`.
2. Form submit adds todo — `crypto.randomUUID()` for id.
3. Map todos to `<li key={id}>` with checkbox and delete button.
4. Toggle: map array replacing matching id immutably.
5. Filter footer optional: all/active/completed counts.
6. Empty state UI when no todos.

Hints

1. `key={todo.id}` never index for dynamic lists.
2. Spread or map for updates: `todos.map(t => t.id === id ? {...t, done: !t.done} : t)`.
3. Prevent empty string submits.

Exact Answer

- Add todo via form → appears in list with unique id
- Toggle checkbox → strikethrough/done state updates immutably
- Delete removes item without page reload
- Empty submit blocked

Complete Solution

```
import { useState, FormEvent } from "react";

interface Todo { id: string; text: string; done: boolean; }

export function TodoList() {
  const [todos, setTodos] = useState<Todo[]>([]);
  const [input, setInput] = useState("");

  const addTodo = (e: FormEvent) => {
    e.preventDefault();
    if (!input.trim()) return;
    setTodos(prev => [...prev, { id: crypto.randomUUID(), text: input.trim(), done: false }]);
    setInput("");
  };

  const toggle = (id: string) =>
    setTodos(prev => prev.map(t => t.id === id ? { ...t, done: !t.done } : t));

  const remove = (id: string) =>
    setTodos(prev => prev.filter(t => t.id !== id));

  return (
    <div>
      <form onSubmit={addTodo}>
        <input value={input} onChange={e => setInput(e.target.value)} />
        <button type="submit">Add</button>
      </form>
      <ul>
        {todos.map(t => (
          <li key={t.id}>
            <input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} />
            <span style={{ textDecoration: t.done ? "line-through" : "none" }}>{t.text}</span>
            <button onClick={() => remove(t.id)}>Delete</button>
          </li>
        ))}
      </ul>
      {todos.length === 0 && <p>No todos yet.</p>}
    </div>
  );
}
```

Expected output / acceptance criteria

- Full CRUD on todos in UI without page reload.

Where to practice

- Local project; React tutorial tic-tac-toe patterns: <https://react.dev/learn/tutorial-tic-tac-toe>

Common mistakes

- Using array index as key after reorder/delete.
- Mutating todo object in place.

80 LPA Interview Twist

Meta live-coding extends Todo to optimistic updates with React Query, undo, and drag-reorder — expect immutable patterns plus performance discussion (virtualization for 10k items).

Interview connection

Classic React live-coding problem; extends to global state weeks later.

Time estimate: 90 minutes

Exercise W5-4: useEffect – Document Title & Fetch

Problem statement

Sync `document.title` with todo count. Fetch posts from JSONPlaceholder on mount, display loading/error/data.

Learning objective

Effect dependencies, cleanup, fetch lifecycle.

Prerequisites

- Theory: [03-react-complete.md §5 useEffect](#)

Step-by-step approach

1. `useEffect(() => { document.title = `Todos: ${count}`; }, [count])`.`
2. Separate effect for fetch: `useState` loading/error/data.
3. `useEffect(() => { fetch(...).then... }, [])` empty deps = mount once.
4. Optional AbortController cleanup on unmount.
5. Render loading spinner, error message, list.

Hints

1. Dependency array omitted = every render (usually wrong).
2. Empty `[]` = mount only.
3. Include all reactive values used inside effect in deps.

Exact Answer

- `document.title` updates to `"Todos: N"` when todo count changes
- Posts fetch once on mount; shows Loading → data or error
- No infinite re-fetch loop

Complete Solution

```
import { useState, useEffect } from "react";

interface Post { id: number; title: string; body: string; }

export function PostsWithTitle({ todoCount }: { todoCount: number }) {
  const [posts, setPosts] = useState<Post[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    document.title = `Todos: ${todoCount}`;
  }, [todoCount]);

  useEffect(() => {
    const controller = new AbortController();
    setLoading(true);
    fetch("https://jsonplaceholder.typicode.com/posts", { signal: controller.signal })
      .then(r => { if (!r.ok) throw new Error("Failed"); return r.json(); })
      .then(setPosts)
      .catch(e => { if (e.name !== "AbortError") setError(e.message); })
      .finally(() => setLoading(false));
    return () => controller.abort();
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return <ul>{posts.slice(0, 5).map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

Expected output / acceptance criteria

- Title updates with todo count; posts load once on mount.

Where to practice

- JSONPlaceholder: <https://jsonplaceholder.typicode.com/posts>
- React useEffect: <https://react.dev/reference/react/useEffect>

Common mistakes

- Infinite loop by setting state in effect without proper deps.
- No loading state UX.

80 LPA Interview Twist

Google asks why React 19 `use()` replaces some `useEffect` fetch patterns, and how to handle race conditions with request IDs — stale closure bugs in effects are a senior-level trap.

Interview connection

"Explain useEffect" + stale closure follow-ups common in React interviews.

Time estimate: 75 minutes

Exercise W5-5: Conditional Rendering Patterns

Problem statement

Demo page showing: `&&` short-circuit, ternary, early return, switch for status enum.

Learning objective

Readable conditional UI without nested ternary hell.

Step-by-step approach

1. Create `StatusBadge` with props `status: 'idle' | 'loading' | 'error' | 'success'`.
2. Implement with switch; map to colors.
3. Parent uses early return for loading full page.
4. Document when to use each pattern in comments.

Exact Answer

- `StatusBadge` renders distinct colors for idle/loading/error/success
- Demo page shows `&&`, ternary, early return, and switch patterns
- Avoid `0 && <Component />` bug — use `count > 0 &&`

Complete Solution

```
type Status = "idle" | "loading" | "error" | "success";

const COLORS: Record<Status, string> = {
  idle: "gray", loading: "blue", error: "red", success: "green",
};

export function StatusBadge({ status }: { status: Status }) {
  switch (status) {
    case "idle": return <span style={{ color: COLORS.idle }}>Idle</span>;
    case "loading": return <span style={{ color: COLORS.loading }}>Loading...</span>;
    case "error": return <span style={{ color: COLORS.error }}>Error</span>;
    case "success": return <span style={{ color: COLORS.success }}>Success</span>;
  }
}

export function ConditionalDemo({ status, count }: { status: Status; count: number }) {
  if (status === "loading") return <p>Loading page...</p>; // early return
  return (
    <div>
      {status === "error" && <p>Something went wrong</p>}
      {count > 0 ? <p>{count} items</p> : <p>No items</p>}
      <StatusBadge status={status} />
    </div>
  );
}
```

Expected output / acceptance criteria

- Four statuses render distinct UI; no runtime errors on falsy `&&` (avoid `0 &&` bug demo).

80 LPA Interview Twist

Meta UI engineers ask you to extract conditional rendering into a polymorphic component library with accessibility — pattern choice affects maintainability at scale.

Where to practice

- <https://react.dev/learn/conditional-rendering>

Time estimate: 40 minutes

Weekend Project W5: Multi-Step Form Wizard

Problem statement

3-step registration wizard: Personal → Account → Review. Persist state across steps; back/next; final submit.

Learning objective

Multi-step state machine, form validation per step, UX polish.

Step-by-step approach

1. State object holding all fields; **step** 1–3.
2. Step components receive slice + update handlers.
3. Validate step before advance (required fields).
4. Review step shows summary; submit logs JSON.
5. Progress indicator UI.
6. Optional: persist draft to sessionStorage.
7. Mobile-responsive layout.

Exact Answer

- 3-step wizard: Personal → Account → Review with data preserved on back navigation
- Invalid step blocks advance; Review shows summary JSON on submit
- Optional sessionStorage draft persistence

Complete Solution

```
import { useState } from "react";

interface FormData { name: string; email: string; password: string; }

export function FormWizard() {
  const [step, setStep] = useState(1);
  const [data, setData] = useState<FormData>({ name: "", email: "", password: "" })

  const update = (fields: Partial<FormData>) => setData(d => ({ ...d, ...fields }))

  const next = () => {
    if (step === 1 && !data.name) return alert("Name required");
    if (step === 2 && (!data.email || data.password.length < 8)) return alert("Invalid email or password");
    setStep(s => s + 1);
  };

  return (
    <div>
      <p>Step {step}/3</p>
      {step === 1 && <input placeholder="Name" value={data.name} onChange={e => update({ name: e.target.value })}>}
      {step === 2 && (<
        <input placeholder="Email" value={data.email} onChange={e => update({ email: e.target.value })}>
        <input type="password" placeholder="Password" value={data.password} onChange={e => update({ password: e.target.value })}>
      </>)}
      {step === 3 && <pre>{JSON.stringify(data, null, 2)}</pre>}
      {step > 1 && <button onClick={() => setStep(s => s - 1)}>Back</button>}
      {step < 3 ? <button onClick={next}>Next</button> : <button onClick={() => console.log(data)}>Done</button>}
    </div>
  );
}
```

Expected output / acceptance criteria

- Navigate back without losing data; invalid step blocked.

Where to practice

- Local: [month-02/week-05/form-wizard/](#)

Interview connection

Checkout flows and onboarding wizards in product companies.

80 LPA Interview Twist

Amazon checkout flows ask for server-side validation per step, progress persistence across tabs, and analytics funnel tracking — wizard state machines mirror production e-commerce.

Time estimate: 5 hours

WEEK 6 – React Hooks Deep (Days 36-42)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Exercise W6-1: Custom Hook useFetch

Problem statement

`useFetch<T>(url)` returns `{ data, loading, error, refetch }`.

Learning objective

Extract effect logic; generic typing.

Prerequisites

- Theory: [03-react-complete.md §6 Custom Hooks](#)

Step-by-step approach

1. Create `hooks/useFetch.ts`.
2. Internal state for data/loading/error.
3. `fetchData` function called on mount and refetch.
4. AbortController on cleanup when url changes.
5. Use in component with `useFetch<Post[]>(url)`.

Hints

1. Reset loading true on refetch.
2. Compare url in deps to refetch when prop changes.
3. Don't fetch if url empty.

Exact Answer

- Returns `{ data: T | null, loading: boolean, error: string | null, refetch }`
- Re-fetches when URL changes; aborts in-flight request on cleanup

Complete Solution

```
import { useState, useEffect, useCallback } from "react";

export function useFetch<T>(url: string) {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const fetchData = useCallback(async (signal?: AbortSignal) => {
    if (!url) return;
    setLoading(true);
    setError(null);
    try {
      const res = await fetch(url, { signal });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      setData(await res.json());
    } catch (e) {
      if ((e as Error).name !== "AbortError") setError((e as Error).message);
    } finally {
      setLoading(false);
    }
  }, [url]);

  useEffect(() => {
    const controller = new AbortController();
    fetchData(controller.signal);
    return () => controller.abort();
  }, [fetchData]);

  return { data, loading, error, refetch: () => fetchData() };
}
```

Expected output / acceptance criteria

- Hook reusable across components; types flow to `data`.

80 LPA Interview Twist

Google expects comparison with React Query/TanStack Query — caching, stale-while-revalidate, and deduplication are the follow-up when your custom hook is "good enough."

Where to practice

- Local: `month-02/week-06/hooks/useFetch.ts`

Time estimate: 60 minutes

Exercise W6-2: useLocalStorage

Problem statement

`useLocalStorage<T>(key, initial)` syncs state with localStorage JSON.

Step-by-step approach

1. Lazy init from localStorage parse or initial.
2. `useEffect` write on state change.
3. Handle SSR: guard `typeof window`.
4. JSON parse error fallback to initial.

Exact Answer

- State syncs to localStorage on every change
- Initial load reads from localStorage or falls back to default
- SSR-safe with `typeof window` guard

Complete Solution

```
import { useState, useEffect } from "react";

export function useLocalStorage<T>(key: string, initial: T) {
  const [value, setValue] = useState<T>(() => {
    if (typeof window === "undefined") return initial;
    try {
      const stored = localStorage.getItem(key);
      return stored ? JSON.parse(stored) : initial;
    } catch {
      return initial;
    }
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue] as const;
}
```

Time estimate: 45 minutes

80 LPA Interview Twist

Meta asks about cross-tab sync via `storage` event and encrypting sensitive localStorage data — privacy and security matter for auth tokens.

Exercise W6-3: useDebounce

Problem statement

`useDebounce(value, delay)` returns debounced value for search inputs.

Step-by-step approach

1. `useState` `debouncedValue`; `useEffect` with `setTimeout`.
2. Cleanup `clearTimeout` on `value/delay` change.
3. Demo: search users after 300ms idle.

Exact Answer

- `useDebounce(value, 300)` returns value only after 300ms of no changes
- Search input demo fires API call once user stops typing

Complete Solution

```
import { useState, useEffect } from "react";

export function useDebounce<T>(value: T, delay: number): T {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}

// Usage: const debouncedQuery = useDebounce(query, 300);
```

Time estimate: 40 minutes

80 LPA Interview Twist

Compare hook-based debounce vs `useDeferredValue` (React 18+) — Google frontend system design for search autocomplete at 1M QPS.

Exercise W6-4: ThemeProvider with Context

Problem statement

Dark/light theme via Context; toggle persists with `localStorage`.

Step-by-step approach

1. `ThemeContext` with `'light' | 'dark'`.
2. Provider wraps app; `useTheme` hook throws if outside provider.
3. CSS variables or class on `document.documentElement`.
4. Persist preference.

Exact Answer

- Theme toggles between light/dark; persists in `localStorage`
- `useTheme()` throws if used outside Provider
- CSS variables update on `document.documentElement`

Complete Solution

```
import { createContext, useContext, useEffect, useState, ReactNode } from "react";

type Theme = "light" | "dark";
const ThemeContext = createContext<{ theme: Theme; toggle: () => void } | null>(null);

export function ThemeProvider({ children }: { children: ReactNode }) {
  const [theme, setTheme] = useState<Theme>(() =>
    (localStorage.getItem("theme") as Theme) || "light"
  );
  useEffect(() => {
    document.documentElement.dataset.theme = theme;
    localStorage.setItem("theme", theme);
  }, [theme]);
  return (
    <ThemeContext.Provider value={{ theme, toggle: () => setTheme(t => t === "light" ? "dark" : "light") }}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error("useTheme must be inside ThemeProvider");
  return ctx;
}
```

Where to practice

- <https://react.dev/learn/passing-data-deeply-with-context>

Time estimate: 75 minutes

80 LPA Interview Twist

Meta design systems use CSS-in-JS vs CSS variables vs Tailwind — expect tradeoff discussion for theming 200+ components without flash-of-wrong-theme.

Exercise W6-5: React Hook Form + Zod Login

Problem statement

Login form: email, password with Zod schema validation and RHF.

Step-by-step approach

1. `npm i react-hook-form zod @hookform/resolvers`.
2. Schema: email format, password min 8.
3. `zodResolver` integration; show field errors.
4. Submit handler mock API call.

Exact Answer

- Email validation and password min 8 chars enforced
- Field-level errors display on blur/submit
- Submit calls mock API with validated data

Complete Solution

```
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";

const LoginSchema = z.object({
  email: z.string().email("Invalid email"),
  password: z.string().min(8, "Min 8 characters"),
});
type LoginForm = z.infer<typeof LoginSchema>;

export function LoginForm() {
  const { register, handleSubmit, formState: { errors } } = useForm<LoginForm>({
    resolver: zodResolver(LoginSchema),
  });

  const onSubmit = async (data: LoginForm) => {
    await fetch("/api/login", { method: "POST", body: JSON.stringify(data) });
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register("email")} placeholder="Email" />
      {errors.email && <span>{errors.email.message}</span>}
      <input {...register("password")} type="password" placeholder="Password" />
      {errors.password && <span>{errors.password.message}</span>}
      <button type="submit">Login</button>
    </form>
  );
}
```

Where to practice

- RHF: <https://react-hook-form.com/>
- Zod: <https://zod.dev/>

Time estimate: 90 minutes

80 LPA Interview Twist

Google asks for accessible form errors (ARIA), server-side validation sync, and preventing double-submit — form handling is a senior frontend signal.

Weekend Project W6: User Directory

Problem statement

Fetch JSONPlaceholder users; search; modal detail view; loading/error states.

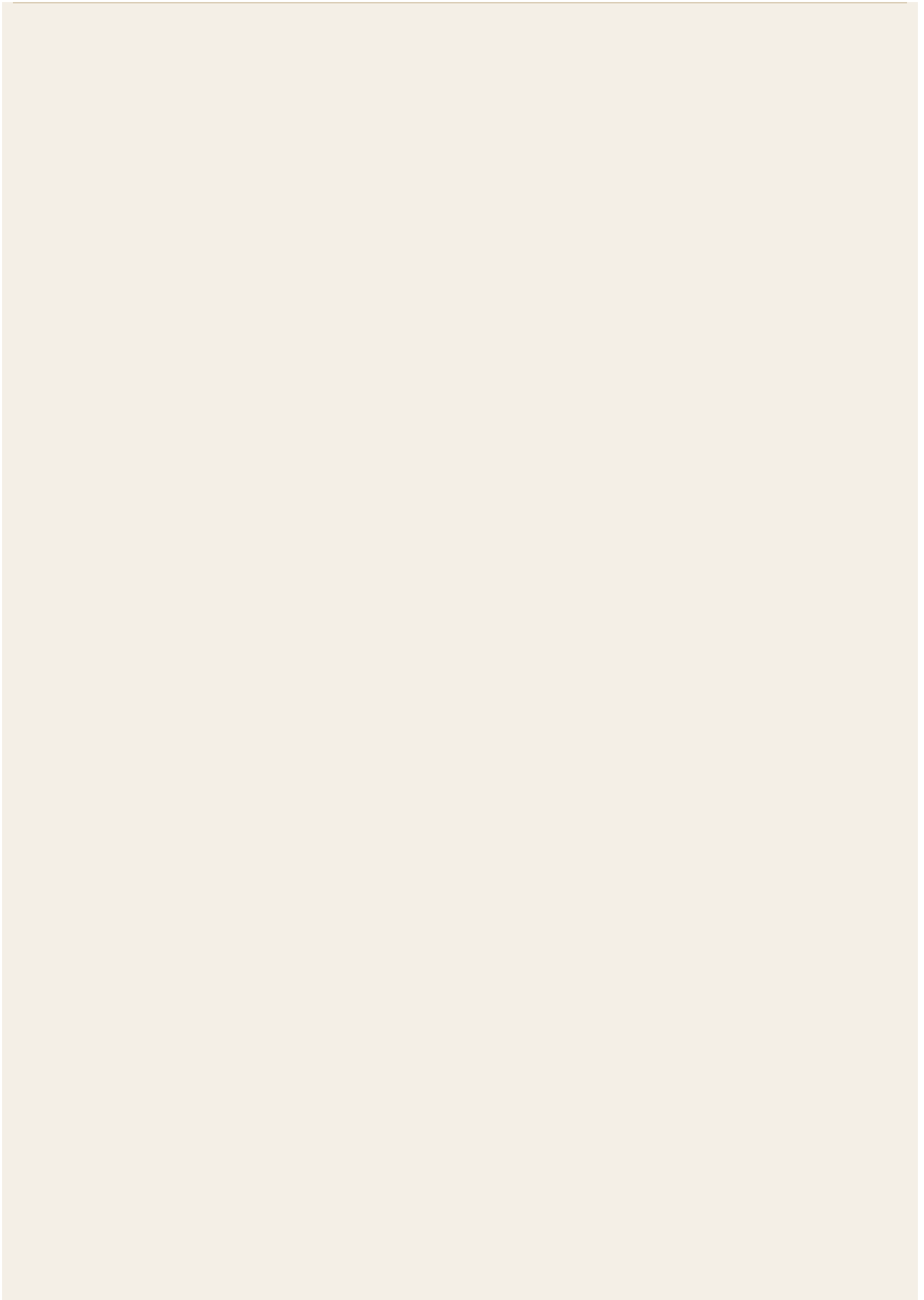
Step-by-step approach

1. List with useFetch; debounced search filters client-side.
2. Modal on click — focus trap optional.
3. useMemo for filtered list.
4. Responsive grid layout.

Exact Answer

- User list loads from JSONPlaceholder with loading/error states
- Debounced search filters by name/email client-side
- Click opens modal with user detail; responsive grid layout

Complete Solution



WEEK 7 – React Advanced + Router (Days 43-49)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Exercise W7-1: React.memo + useMemo Optimization

Problem statement

Parent re-renders often; expensive child list should skip unnecessary renders.

Step-by-step approach

1. Create heavy child with React.memo.
2. Pass stable callback via useCallback.
3. useMemo for sorted/filtered derived data.
4. React DevTools Profiler before/after demo.

Exact Answer

- `React.memo` child skips re-render when props unchanged (shallow compare)
- `useCallback` stabilizes function references passed to memoized children
- `useMemo` caches expensive sorted/filtered computation

Complete Solution

```
import { memo, useMemo, useCallback, useState } from "react";

const ExpensiveList = memo(function ExpensiveList({
  items, onSelect,
}): { items: string[]; onSelect: (item: string) => void } {
  console.log("ExpensiveList render");
  return <ul>{items.map(i => <li key={i} onClick={() => onSelect(i)}>{i}</li>)}</ul>
});

export function OptimizedParent({ data }: { data: number[] }) {
  const [count, setCount] = useState(0);
  const sorted = useMemo(() => [...data].sort((a, b) => a - b).map(String), [data])
  const handleSelect = useCallback((item: string) => console.log(item), []);
  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>Re-render {count}</button>
      <ExpensiveList items={sorted} onSelect={handleSelect} />
    </div>
  );
}
```

Where to practice

- <https://react.dev/reference/react/memo>

Time estimate: 60 minutes

80 LPA Interview Twist

Google React perf interviews ask "when NOT to memo" — over-memoization hurts; expect React Compiler (React 19) discussion and Profiler-driven optimization.

Exercise W7-2: Lazy Routes + Suspense

Problem statement

Code-split route components with `React.lazy` and Suspense fallback.

Step-by-step approach

1. `const About = lazy(() => import('./pages/About'))`.
2. Wrap Routes in Suspense with spinner.
3. Verify separate chunks in Vite build output.

Exact Answer

- Route components load as separate JS chunks on navigation

- Suspense shows fallback spinner during chunk load
- Vite build output shows split chunks per lazy route

Complete Solution

```
import { lazy, Suspense } from "react";
import { BrowserRouter, Routes, Route } from "react-router-dom";

const Home = lazy(() => import("./pages/Home"));
const About = lazy(() => import("./pages/About"));

export function App() {
  return (
    <BrowserRouter>
      <Suspense fallback=<div>Loading...</div>>
        <Routes>
          <Route path="/" element=<Home /> />
          <Route path="/about" element=<About /> />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}
```

Time estimate: 45 minutes

80 LPA Interview Twist

Meta asks about prefetching lazy routes on hover and SSR with React.lazy limitations — code splitting strategy affects LCP and TTI metrics.

Exercise W7-3: React Router v6 – Four Routes + 404

Problem statement

Routes: Home, About, User/:id, NotFound catch-all.

Step-by-step approach

1. `npm i react-router-dom`.
2. `createBrowserRouter` or `BrowserRouter` + `Routes`.
3. `useParams` on User detail; invalid id handling.
4. `NavLink` active styles.

Exact Answer

- Routes: `/`, `/about`, `/user/:id`, `*` (404)
- `useParams()` extracts user id on detail page
- `NavLink` applies active class on current route

Complete Solution

```
import { BrowserRouter, Routes, Route, NavLink, useParams } from "react-router-dom"

function UserDetails() {
  const { id } = useParams();
  return <h1>User {id}</h1>;
}

function NotFound() { return <h1>404 Not Found</h1>; }

export function RouterApp() {
  return (
    <BrowserRouter>
      <nav>
        <NavLink to="/" end>Home</NavLink>
        <NavLink to="/about">About</NavLink>
      </nav>
      <Routes>
        <Route path="/" element={<h1>Home</h1>} />
        <Route path="/about" element={<h1>About</h1>} />
        <Route path="/user/:id" element={<UserDetail />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Where to practice

- <https://reactrouter.com/en/main/start/tutorial>

Time estimate: 75 minutes

80 LPA Interview Twist

Google asks about data loaders (React Router 6.4+), nested layouts, and route-based code splitting with authentication — routing architecture for large SPAs.

Exercise W7-4: Protected Route

Problem statement

Redirect to `/login` if not authenticated (mock boolean in context).

Step-by-step approach

1. AuthContext with `isLoggedIn`, `login`, `logout`.
2. `ProtectedRoute` wrapper using `Navigate` from react-router.
3. Preserve intended URL in location state optional.

Exact Answer

- Unauthenticated user visiting `/dashboard` redirects to `/login`
- After login, optionally redirect back to intended URL
- Auth state managed via Context

Complete Solution

```
import { createContext, useContext, useState, ReactNode } from "react";
import { Navigate, useLocation } from "react-router-dom";

const AuthContext = createContext({ isLoggedIn: boolean; login: () => void; logout: () => void });

export function AuthProvider({ children }: { children: ReactNode }) {
  const [isLoggedIn, setIsLoggedIn] = useState(false);
  return (
    <AuthContext.Provider value={{ isLoggedIn, login: () => setIsLoggedIn(true), logout: () => setIsLoggedIn(false) }}>
      {children}
    </AuthContext.Provider>
  );
}

export function ProtectedRoute({ children }: { children: ReactNode }) {
  const auth = useContext(AuthContext);
  const location = useLocation();
  if (!auth.isLoggedIn) return <Navigate to="/login" state={{ from: location }} replace />;
  return <>{children}</>;
}
```

Time estimate: 50 minutes

80 LPA Interview Twist

Meta production auth uses token refresh, role-based route guards, and SSR cookie validation in Next.js middleware — client-only boolean checks are insufficient at scale.

Exercise W7-5: Error Boundary

Problem statement

Class or library error boundary catches render errors; shows fallback UI.

Step-by-step approach

1. `componentDidCatch` + `getDerivedStateFromError`.
2. Wrap risky component tree.
3. Reset button clears error state.

Exact Answer

- Render errors in child tree caught by Error Boundary
- Fallback UI shown instead of crashed component tree
- Reset button recovers without full page reload

Complete Solution

```
import { Component, ReactNode } from "react";

interface State { hasError: boolean; }

export class ErrorBoundary extends Component<{ children: ReactNode; fallback?: ReactNode; state: State = { hasError: false } > {

  static getDerivedStateFromError(): State {
    return { hasError: true };
  }

  componentDidCatch(error: Error, info: React.ErrorInfo) {
    console.error("ErrorBoundary caught:", error, info);
  }

  render() {
    if (this.state.hasError) {
      return this.props.fallback ?? (
        <div>
          <h2>Something went wrong.</h2>
          <button onClick={() => this.setState({ hasError: false })}>Try again</button>
        </div>
      );
    }
    return this.props.children;
  }
}
```

Where to practice

- <https://react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary>

Time estimate: 45 minutes

80 LPA Interview Twist

Google asks about error boundaries vs try/catch, Sentry integration, and React 19's improved error handling — boundaries don't catch event handler or async errors.

Weekend Project W7: E-Commerce Listing + Cart

Problem statement

Mock product API; filter/sort; cart in Zustand or Context; persist cart.

Step-by-step approach

1. Product grid with categories filter.
2. Add to cart; cart drawer with quantities.
3. Zustand store: `items`, `addItem`, `removeItem`, `total`.
4. `localStorage` persist middleware.

Exact Answer

- Product grid with category filter and sort
- Cart drawer shows items, quantities, and total
- Zustand store persists cart to `localStorage` across sessions

Complete Solution

```
import { create } from "zustand";
import { persist } from "zustand/middleware";

interface CartItem { id: number; title: string; price: number; qty: number; }

interface CartStore {
  items: CartItem[];
  addItem: (product: Omit<CartItem, "qty">) => void;
  removeItem: (id: number) => void;
  total: () => number;
}

export const useCart = create<CartStore>()(
  persist(
    (set, get) => ({
      items: [],
      addItem: (product) => set(state => {
        const existing = state.items.find(i => i.id === product.id);
        if (existing) return { items: state.items.map(i => i.id === product.id ? {
          return { items: [...state.items, { ...product, qty: 1 }] };
        }},
      removeItem: (id) => set(state => ({ items: state.items.filter(i => i.id !==
      total: () => get().items.reduce((sum, i) => sum + i.price * i.qty, 0),
    },
    { name: "cart-storage" }
  )
);
```

Where to practice

- Zustand: <https://zustand.docs.pmnd.rs/>
- Fake Store API: <https://fakestoreapi.com/products>

Time estimate: 6 hours

80 LPA Interview Twist

Amazon frontend system design extends this to optimistic cart sync, inventory reservation, and conflict resolution when price changes — Zustand vs Redux Toolkit vs React Query for cart state.

WEEK 8 – Next.js (Days 50-56)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Exercise W8-1: Next.js 14 App Router Setup

Problem statement

Create Next.js app with App Router, TypeScript, Tailwind.

Step-by-step approach

1. `npx create-next-app@latest blog --ts --tailwind --app`.
2. Explore `app/layout.tsx`, `app/page.tsx`.
3. Run dev server port 3000.

Exact Answer

- Next.js app runs at `http://localhost:3000`
- App Router structure: `app/layout.tsx`, `app/page.tsx`
- TypeScript + Tailwind configured

Complete Solution

```
npx create-next-app@latest blog --ts --tailwind --app --eslint
cd blog && npm run dev
```

```
// app/layout.tsx
export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body className="min-h-screen bg-white text-gray-900">{children}</body>
    </html>
  );
}

// app/page.tsx
export default function Home() {
  return <main className="p-8"><h1 className="text-3xl font-bold">Blog</h1></main>
}
```

Where to practice

- <https://nextjs.org/docs/app/getting-started/installation>

Time estimate: 30 minutes

80 LPA Interview Twist

Meta/Vercel teams ask App Router vs Pages Router migration tradeoffs, and when to use monorepo with Turborepo — framework choice affects team velocity at 80 LPA companies.

Exercise W8-2: Server Component Data Fetch

Problem statement

Blog index page fetches posts in Server Component (async function component).

Step-by-step approach

1. Create `lib/posts.ts` fetching from CMS or MDX filesystem.
2. Page component async — no `useEffect`.
3. Discuss caching: `fetch` options, `revalidate`.

Exact Answer

- Blog index renders posts fetched on server — no client `useEffect`
- HTML includes post data on first paint (SEO-friendly)
- `fetch` with `{ next: { revalidate: 3600 } }` enables ISR

Complete Solution

```
// lib/posts.ts
export async function getPosts() {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts", {
    next: { revalidate: 3600 },
  });
  return res.json() as Promise<{ id: number; title: string; body: string }[]>;
}

// app/blog/page.tsx — Server Component
import { getPosts } from "@lib/posts";

export default async function BlogPage() {
  const posts = await getPosts();
  return (
    <ul>
      {posts.slice(0, 10).map(p => (
        <li key={p.id}><h2>{p.title}</h2><p>{p.body.slice(0, 100)}...</p></li>
      ))}
    </ul>
  );
}
```

Where to practice

- <https://nextjs.org/docs/app/building-your-application/data-fetching>

Time estimate: 60 minutes

80 LPA Interview Twist

Google/Shopify ask about streaming SSR with Suspense boundaries, partial prerendering (Next.js 15), and cache invalidation strategies — server components change data fetching architecture.

Exercise W8-3: Client Component in Server Page

Problem statement

Embed interactive counter (`'use client'`) inside server-rendered page.

Step-by-step approach

1. Mark counter with `'use client'` directive.
2. Import into server page — boundary understood.
3. Explain bundle split implications.

Exact Answer

- Server page renders static content; interactive counter hydrates on client

- `'use client'` directive marks client boundary
- Counter JS ships as separate client bundle chunk

Complete Solution

```
// app/components/Counter.tsx
"use client";
import { useState } from "react";
export function Counter() {
  const [n, setN] = useState(0);
  return <button onClick={() => setN(n => n + 1)}>Count: {n}</button>;
}

// app/page.tsx – Server Component
import { Counter } from "../components/Counter";
export default function Page() {
  return (
    <main>
      <h1>Server-rendered heading</h1>
      <Counter />
    </main>
  );
}
```

Time estimate: 40 minutes

80 LPA Interview Twist

Meta frontend architecture reviews push client boundaries as low as possible — expect discussion of bundle size impact and React Server Components composition patterns.

Exercise W8-4: Dynamic Route `/blog/[slug]`

Problem statement

Generate static params for posts; render MDX or markdown content.

Step-by-step approach

1. `app/blog/[slug]/page.tsx` .
2. `generateStaticParams` for slugs.
3. `notFound()` for invalid slug.

Exact Answer

- `/blog/hello-world` renders post content from MDX/markdown

- `generateStaticParams` pre-builds known slugs at build time
- Invalid slug triggers Next.js 404 page

Complete Solution

```
// app/blog/[slug]/page.tsx
import { notFound } from "next/navigation";

const posts: Record<string, { title: string; content: string }> = {
  "hello-world": { title: "Hello World", content: "First post content..." },
  "nextjs-tips": { title: "Next.js Tips", content: "Tips content..." },
};

export function generateStaticParams() {
  return Object.keys(posts).map(slug => ({ slug }));
}

export default function PostPage({ params }: { params: { slug: string } }) {
  const post = posts[params.slug];
  if (!post) notFound();
  return <article><h1>{post.title}</h1><p>{post.content}</p></article>;
}
```

Time estimate: 90 minutes

80 LPA Interview Twist

Vercel/Meta ask about on-demand ISR vs SSG for CMS content, and edge-rendered dynamic routes — slug routing at scale involves CDN cache keys and revalidation webhooks.

Exercise W8-5: API Route POST /api/contact

Problem statement

Route handler validates body with Zod; returns 201 or 400.

Step-by-step approach

1. `app/api/contact/route.ts` export POST.
2. Parse JSON; Zod schema name/email/message.
3. Mock email send; log payload.

Exact Answer

- Valid body → 201 `{ success: true }`
- Invalid body → 400 with Zod error details

- Malformed JSON → 400 error response

Complete Solution

```
// app/api/contact/route.ts
import { NextRequest, NextResponse } from "next/server";
import { z } from "zod";

const ContactSchema = z.object({
  name: z.string().min(1),
  email: z.string().email(),
  message: z.string().min(10),
});

export async function POST(req: NextRequest) {
  try {
    const body = await req.json();
    const parsed = ContactSchema.safeParse(body);
    if (!parsed.success) {
      return NextResponse.json({ errors: parsed.error.issues }, { status: 400 });
    }
    console.log("Contact form:", parsed.data);
    return NextResponse.json({ success: true }, { status: 201 });
  } catch {
    return NextResponse.json({ error: "Invalid JSON" }, { status: 400 });
  }
}
```

Where to practice

- <https://nextjs.org/docs/app/building-your-application/routing/route-handlers>

Time estimate: 50 minutes

80 LPA Interview Twist

Production contact APIs need rate limiting, CAPTCHA, honeypot fields, and idempotency keys — Google security review catches missing validation on public endpoints.

Weekend Project W8: Blog + Deploy

Problem statement

5 MDX posts, SEO metadata, OG tags, deploy to Vercel.

Step-by-step approach

1. `@next/mdx` setup; write 5 posts.

2. `generateMetadata` per post.
3. Sitemap optional.
4. `vercel deploy` — live URL on resume.

Exact Answer

- 5 MDX blog posts with SEO metadata and OG tags
- Deployed live on Vercel with custom domain optional
- Lighthouse SEO score ≥ 90

Complete Solution

```
// app/blog/[slug]/page.tsx - with metadata
import type { Metadata } from "next";

export async function generateMetadata({ params }: { params: { slug: string } }):
  const post = getPost(params.slug);
  return {
    title: post.title,
    description: post.excerpt,
    openGraph: { title: post.title, description: post.excerpt, type: "article" },
  };
}
```

```
npm i @next/mdx @mdx-js/react
# Write 5 posts in content/posts/*.mdx
vercel deploy --prod
```

Where to practice

- Next.js Learn: <https://nextjs.org/learn>
- Vercel: <https://vercel.com/docs>

Time estimate: 6 hours

80 LPA Interview Twist

Resume project review at Meta/Google includes Core Web Vitals, edge caching strategy, and cost analysis of Vercel bandwidth — deployed blog must demonstrate production engineering, not just tutorial completion.

Month 2 LeetCode Reference (Weeks 5-8)

Week	Problems	Links
W5	LC 21, 26, 27, 35, 58	https://leetcode.com/problemset/
W6	LC 66, 67, 69, 70, 83	Same
W7	LC 88, 94, 100, 101, 104	Same
W8	LC 108, 110, 111, 112, 118	Same

For each LC problem, follow template: brute force → optimal → complexity → edge cases (use Week 1 detailed format).

Month 2 Completion Checklist

Milestone	Done
Vite React TS app	☐
Todo + hooks + context	☐
User directory + form wizard	☐
Router + protected routes + e-commerce	☐
Next.js blog deployed	☐
60+ LeetCode total	☐

Total estimated time: ~80–96 hours (Days 29–56)